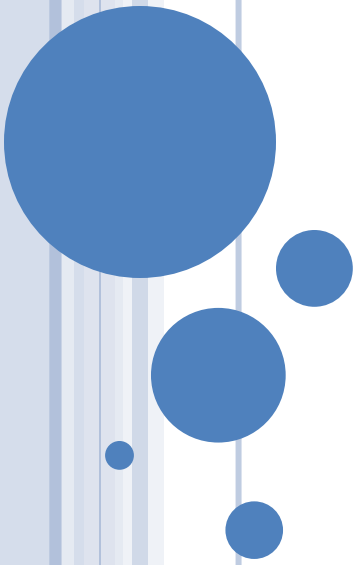
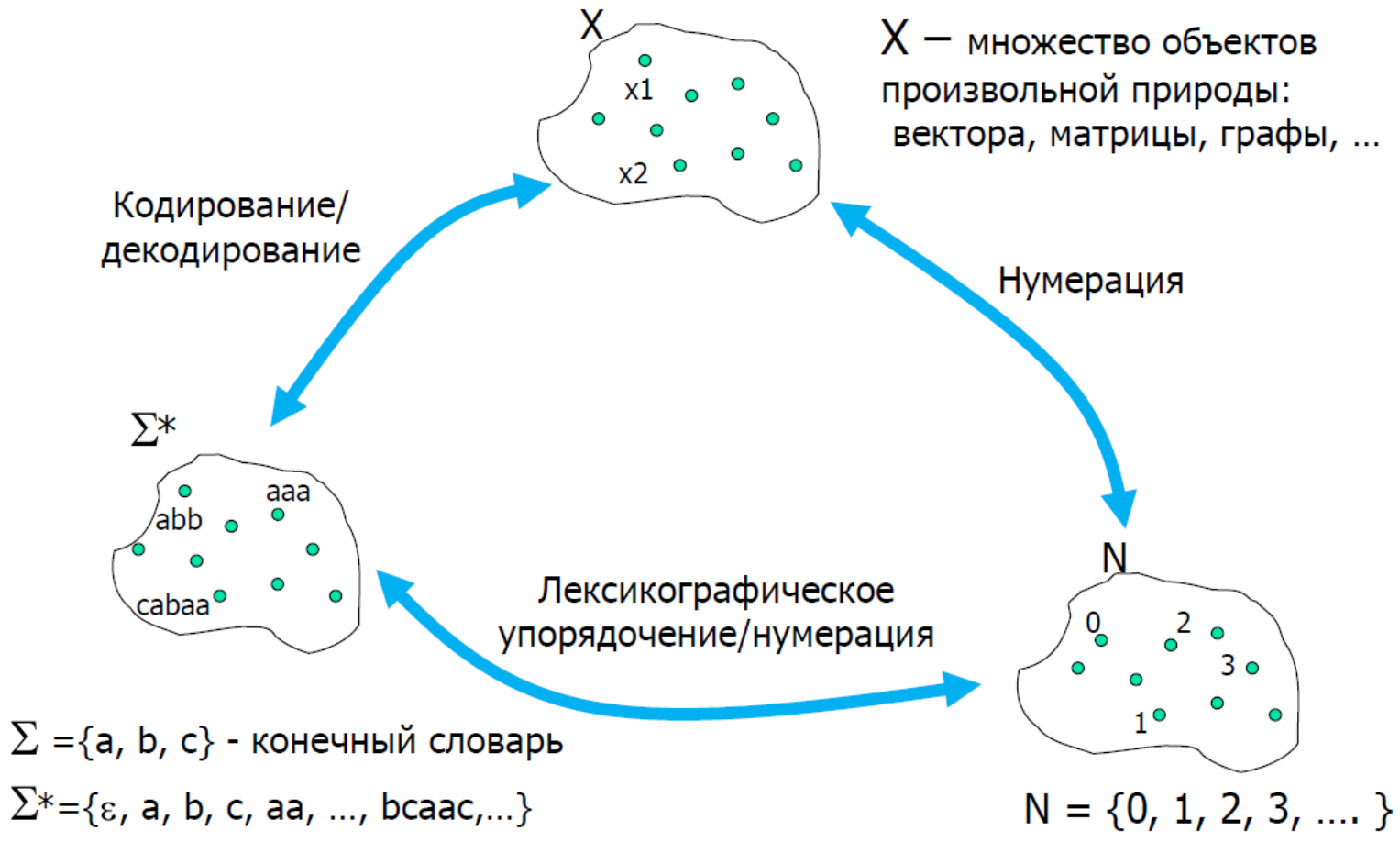


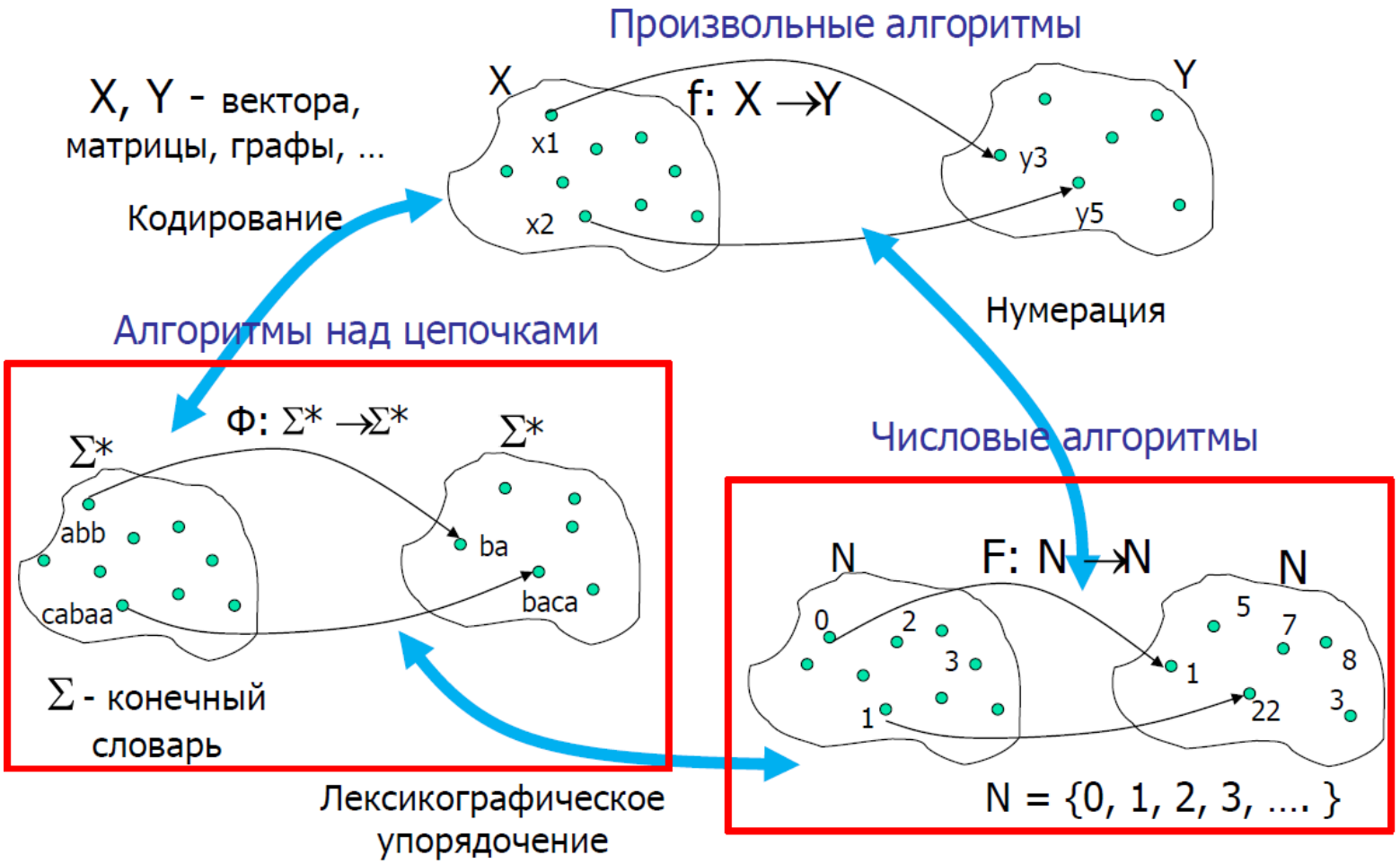
Математическая логика и теория алгоритмов

Рекурсивные функции, представление цепочек словаря натуральными числами

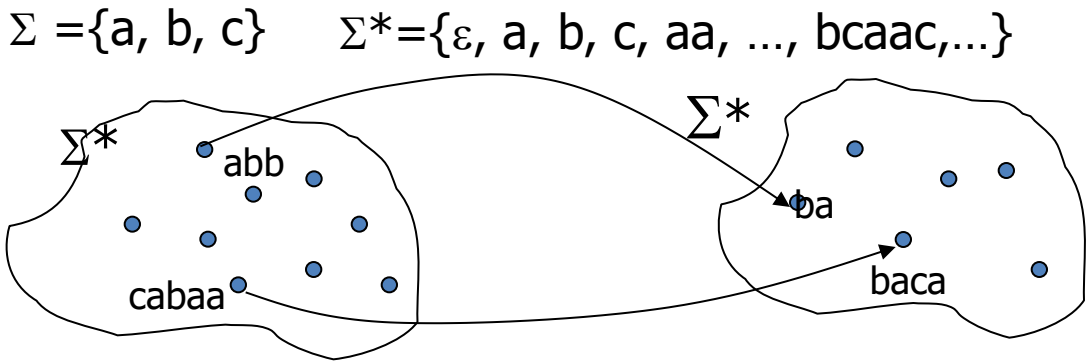
Как однотипно задать элементы множества аргументов и множества результатов в алгоритмах?



Анализ алгоритмов над произвольными структурами данных



Потенциально бесконечные входы



- Для алгоритма не устанавливается граница размера входа. Обрабатываются только входы конечной длины (конечные цепочки). Но для любого заданного натурального N найдется цепочка длиной большей, чем N.
- Если существует алгоритм, вычисляющий функцию F, то F называется алгоритмически вычислимой функцией (при некотором формальном определении алгоритма.
- Существует несколько определений, позволяющих построить функции как суперпозиции нескольких базовых простых функций. Такие функции называются ***вычислимыми*** (в отличие от алгоритмически вычислимых, т.е. функций, которые вычисляются алгоритмами.
- Тезис Черча: Класс вычислимых функций совпадает с классом алгоритмически вычислимых функций.

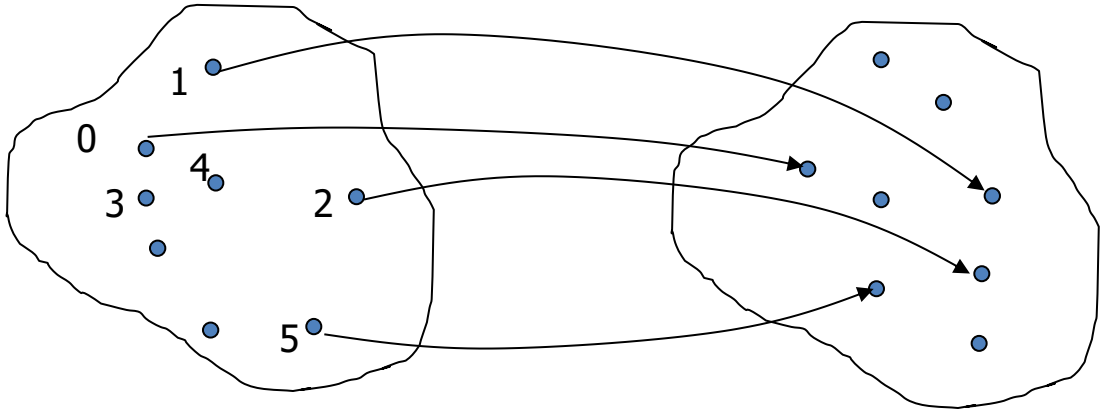
Разрешимые и перечислимые множества

- Множество S называется **алгоритмически разрешимым**, если существует такая машина Тьюринга M, что:
  - $x \in S$ ттогда, когда M(x) останавливается в состоянии q_{yes}
  - $x \notin S$ ттогда, когда M(x) останавливается в состоянии q_{no}
- Множество S называется **перечислимым**, если существует такая машина Тьюринга M, что:
  - $x \in S$ ттогда, когда M(x) останавливается в состоянии q_{yes}
  - $x \notin S$ ттогда, когда M(x) НЕ останавливается



Вычислимые функции. Перечислимые множества

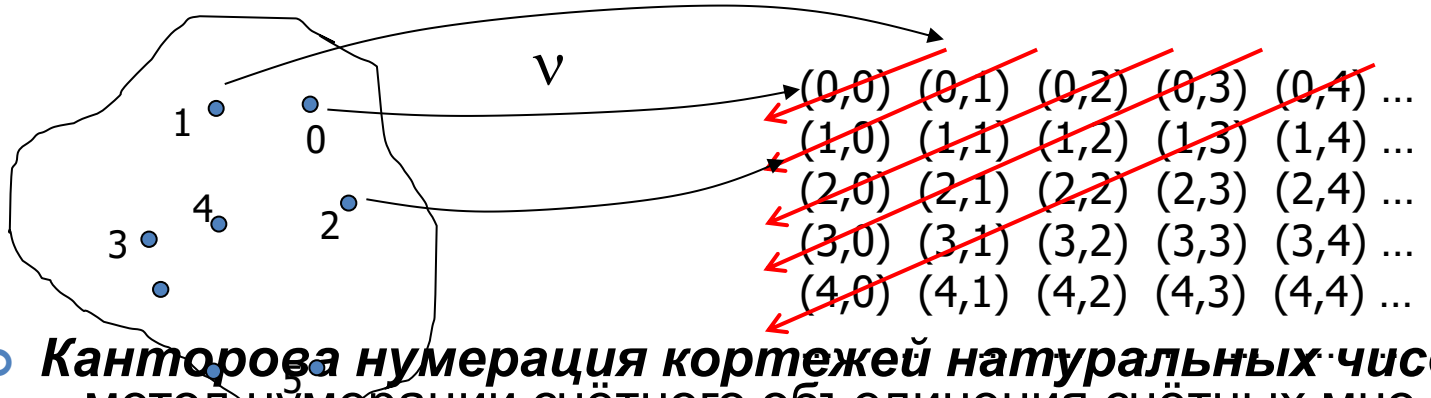
- Определение. Функция, для которой существует вычисляющий ее алгоритм, называется **вычислимой**.
- Для каждой вычислимой функции можно указать два алфавита: *алфавит ее аргументов* (слов в первом алфавите) и *ее результатов* (слов во втором алфавите).
- Определение. **Числовыми функциями** называются такие функции, аргументы которых – натуральные числа. Числа всегда представляются в каком-либо алфавите (унарная запись, двоичная и т.п.).
- Определение. Множество называется **перечислимым**, если оно либо пусто, либо является множеством элементов какой-нибудь вычислимой функции.



Перечислимые множества

ν – функция нумерации

- *Нумерацией множества S* называется всякое отображение ν множества N натуральных чисел на множество S.
- ***Утверждение.*** Любое множество, для которого существует нумерация, перечислимо.
- ***Пример 1.*** Множество N^2 всех пар натуральных чисел перечислимо.



- ***Канторова нумерация кортежей натуральных чисел*** – метод нумерации счётного объединения счётных множеств (Георг Фердинанд Людвиг Филипп Кантор).

0 1 2 3 4 5 6 7 8 9 ...

(0,0) (0,1) (1,0) (0,2) (1,1) (2,0) (0,3) (1,2) (2,1) (3,0) ...

$\nu^{-1}(a,b) = ((a+b)^2 + 3a + b)/2$; $\nu^{-1}(0,2)=3$; $\nu^{-1}(2,1)=8$ ...

Множество Σ^* всех слов в конечном алфавите

Определение (канторовская нумерующая функция).

- Рассмотрим линейный порядок $\prec$ на N^2, определённый следующим образом: $(x1, y1) \prec (x2, y2)$ тогда и только тогда, когда либо $x1+y1 < x2+y2$, либо $x1+y1 = x2+y2$ и $x1 < x2$. Очевидно, $0, 0 \prec 0, 1 \prec 1, 0 \prec 0, 2 \prec 1, 1 \prec 2,$ $0 \prec 0, 3 \prec \ldots$.
- Легко проверить, что для любой пары $x, y \in N^2$ множество пар, меньших чем x, y (в смысле отношения $\prec$), конечно.
- Следовательно, функция из N^2 в N, ставящая каждой паре из N^2 в соответствие количество пар, меньших её (в смысле отношения $\prec$), является биекцией.
- Эта функция называется *канторовской нумерующей функцией* и обозначается c.
- **Пример.** $c\,(0, 0) = 0$, $c\,(0, 1) = 1$, $c\,(1, 0) = 2$.

Множество Σ^* всех слов в конечном алфавите

- Пример 2. Множество Σ^* всех слов в конечном алфавите Σ перечислимо.

- Один из возможных способов нумерации – лексикографическое упорядочение слов:
  - Элементы Σ мощностью k произвольным образом кодируются 1, 2, …, k
    Пример: Пусть Σ ={a, b, c}. Буквы a↔1, b↔2, c↔3
    Упорядочение слов: ε↔0, a↔1, b↔2, c↔3, aa↔4, ab↔5, ac↔6, ba↔7, bb↔8, …
  - Лексикографический номер r_σ слова $\sigma = a_i a_j \dots a_m$ будет равен $k^{m-1}i + k^{m-2}j + \dots + m$

Пример.

- Номера слов: σ_1=aac ; $r_{\sigma 1} = 3^2 \cdot 1 + 3^1 \cdot 1 + 3^0 \cdot 3 = 15$;

- σ_2=cbacb; $r_{\sigma 2} = 3^4 \cdot 3 + 3^3 \cdot 2 + 3^2 \cdot 1 + 3^1 \cdot 3 + 3^0 \cdot 2 = 317$

Алгоритм

- Алгоритм нахождения слова σ из символов конечного алфавита Σ по его номеру r_σ
  - из r_σ вычитается число $z_0 \in \{1, 2, \ldots, k\}$ так, чтобы $r_\sigma - z_0$ делилось на $|\Sigma|=k$.
    $r_1 = (r_\alpha - z_0) / k$
  - из r_1 вычитается число $z_1 \in \{1, 2, \ldots, k\}$ так, чтобы $r_1 - z_1$ делилось на k.
    $r_2 = (r_1 - z_1) / k$
  - Повторяем до возможности деления на k.
- Пример: в алфавите $\Sigma = \{a,b,c\}$ ($k=|\Sigma|=3$) найдем слово, лексикографический номер которого $r_\sigma = 317$.
  - Согласно алгоритму, имеем:

$r_\sigma = 317 = 3*105+2;$	$z_0=2$ (b);	$r_1=105$
$r_1 = 105 = 3*\ 34+3;$	$z_1=3$ (c);	$r_2=\ 34$
$r_2 = 34 = 3*\ 11+1;$	$z_2=1$ (a);	$r_3=\ 11$
$r_3 = 11 = 3*\ \ 3+2;$	$z_3=2$ (b);	$r_4=\ \ 3$
$r_4 = \ \ 3 = 3*\ \ 0+3;$	$z_4=3$ (c);	$r_5=\ \ 0$

  - Следовательно, $317 = 3^4 \cdot 3 + 3^3 \cdot 2 + 3^2 \cdot 1 + 3^1 \cdot 3 + 3^0 \cdot 2 = \leftrightarrow$ слово **cbacb**

Множество Σ^* всех слов в конечном алфавите

Пример 3. Множество Σ^* всех слов в конечном алфавите Σ перечислимо

- Очевидно, что в свете изложенного все множество слов конечной длины A* в алфавите A счетно, то есть $|A^*|=|N|= \lambda'_0$.

Алгоритм нахождения слова по его номеру.

- Этот алгоритм следует из однозначности разложения лексикографического номера l_α искомого слова α по степеням мощности алфавита A, то есть : $l_\alpha =(k_i^{m-1}+ k_j^{m-2}+...+kz)+z_0$, $z_0 \in A$.

Шаги алгоритма:

- Из заданного натурального числа l_α вычитается число z_0 такое, чтобы $l_\alpha - z_0$ делилось на $|A|=k$,
- Из числа $(l_\alpha - z_0)/k$ вычитается слово $z_1 \in A$, такое, чтобы $(l_\alpha - z_0)/k - z_1$ делилось на k,
- Повторять шаг 2 до возможности деления на k.

Пример: в алфавите A={a,b,c} найти слово, лексигографический номер которого 311.

- Согласно алгоритму имеем: $l_\alpha =311=309+2=(3\cdot103)+2=(102+1)\cdot3^1+2=3^2\cdot34 + 1\cdot3+2=(33+1)\cdot3^2+1\cdot3+2=11\cdot3^3+1\cdot3^2+1\cdot3+2=3\cdot3^4+ 2\cdot3^3+1\cdot3^2+1\cdot3+2$ cbaab.

Лексикографическое упорядочение множеств конечных цепочек

- Лексикографическое упорядочение – нумерация множества Σ^* всех слов конечного словаря (возможный способ нумерации)
  - Элементы Σ мощностью k произвольным образом кодируются 1, 2, …, k

Пример: Пусть $\Sigma = \{a, b, c\}$.

- Кодирование символов : $\chi(a)=1,\ \chi(b)=2,\ \chi(c)=3$
- Лексикографический номер r_σ слова $\sigma = x_{m-1}\, x_{m-2}\, \ldots\, x_0$

$$r_\sigma = k^{m-1}\chi(x_{m-1}) + k^{m-2}\chi(x_{m-2}) + \ldots + k^0\chi(x_0)$$

$\sigma = c\ b\ a\ c\ b$

$r_\sigma = \chi(c)\ \chi(b)\ \chi(a)\ \chi(c)\ \chi(b) =$

$= 3\ 2\ 1\ 3\ 2$ в k-ичной системе счисления

$r_\sigma = 3^4\cdot 3 + 3^3\cdot 2 + 3^2\cdot 1 + 3^1\cdot 3 + 3^0\cdot 2 = 317$

Лексикографическое упорядочение всех слов конечного словаря

Словарь $\Sigma = \{a, b, c\}$:

$\varepsilon \leftrightarrow 0$	$ca \leftrightarrow 10$	$acb \leftrightarrow 20$	$bcc \leftrightarrow 30$	$aaaa \leftrightarrow 40$
$a \leftrightarrow 1$	$cb \leftrightarrow 11$	$acc \leftrightarrow 21$	$caa \leftrightarrow 31$	$aaab \leftrightarrow 41$
$b \leftrightarrow 2$	$cc \leftrightarrow 12$	$baa \leftrightarrow 22$	$cab \leftrightarrow 32$	$aaac \leftrightarrow 42$
$c \leftrightarrow 3$	$aaa \leftrightarrow 13$	$bab \leftrightarrow 23$	$cac \leftrightarrow 33$	$aaba \leftrightarrow 43$
$aa \leftrightarrow 4$	$aab \leftrightarrow 14$	$bac \leftrightarrow 24$	$cba \leftrightarrow 34$	$aabb \leftrightarrow 44$
$ab \leftrightarrow 5$	$aac \leftrightarrow 15$	$bba \leftrightarrow 25$	$cbb \leftrightarrow 35$	$aabc \leftrightarrow 45$
$ac \leftrightarrow 6$	$aba \leftrightarrow 16$	$bbb \leftrightarrow 26$	$cbc \leftrightarrow 36$	$aaca \leftrightarrow 46$
$ba \leftrightarrow 7$	$abb \leftrightarrow 17$	$bbc \leftrightarrow 27$	$cca \leftrightarrow 37$	$aacb \leftrightarrow 47$
$bb \leftrightarrow 8$	$abc \leftrightarrow 18$	$bca \leftrightarrow 28$	$ccb \leftrightarrow 38$	$aacc \leftrightarrow 48$
$bc \leftrightarrow 9$	$aca \leftrightarrow 19$	$bcb \leftrightarrow 29$	$ccc \leftrightarrow 39$	$abaa \leftrightarrow 49$

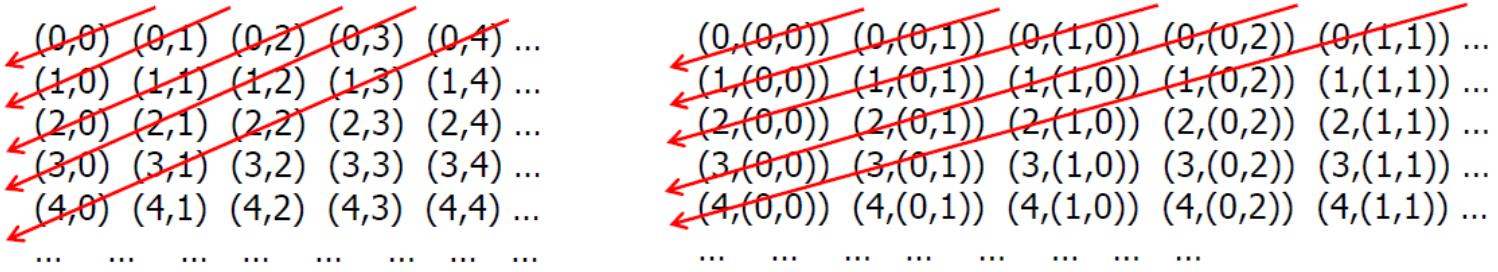
$r_{bac} = \chi(b)\ \chi(a)\ \chi(c) = 2\ 1\ 3$ (в троичной системе) =

$3^2 \cdot 2 + 3^1 \cdot 1 + 3^0 \cdot 3 = 18 + 3 + 3 = 24$

Обобщение канторовой нумерации на нумерацию троек, четверок и т.п.

- $Cantor_1(a_1) = a_1;$
  $Cantor_{n+1}(a_1, \ldots, a_{n+1}) = Cantor_n(a_1, \ldots a_{n-1}, Cantor(a_n, an_{+1}))$

Пример: нумерация троек - $Cantor_3(a_1, a_2, a_3)$



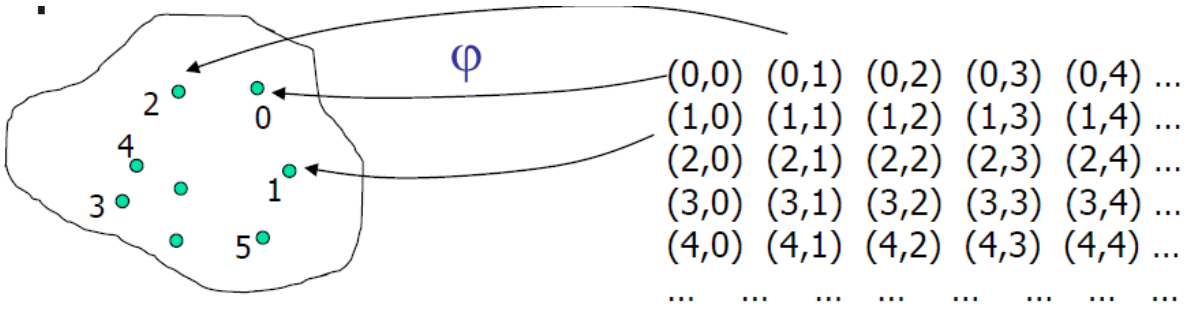
0	1	2	3	4	5	6	7	8	9 ...	Нумерация пар N^2
(0,0)	(0,1)	(1,0)	(0,2)	(1,1)	(2,0)	(0,3)	(1,2)	(2,1)	(3,0) ...	

0	1	2	3	4	5	6	7	8	9 ...	Нумерация троек N^3
(0,0,0)	(0,0,1)	(1,0,0)	(0,1,0)	(1,0,1)	(2,0,0)	(0,0,2)	(1,1,0)	(2,0,1)	(3,0,0) ...	

- Графы, матрицы, вектора, формулы, графики, ... любые структуры данных из элементов мощности не более, чем счетной, можно пронумеровать

Другое представление элементов множества натуральными числами

φ – функция нумерации

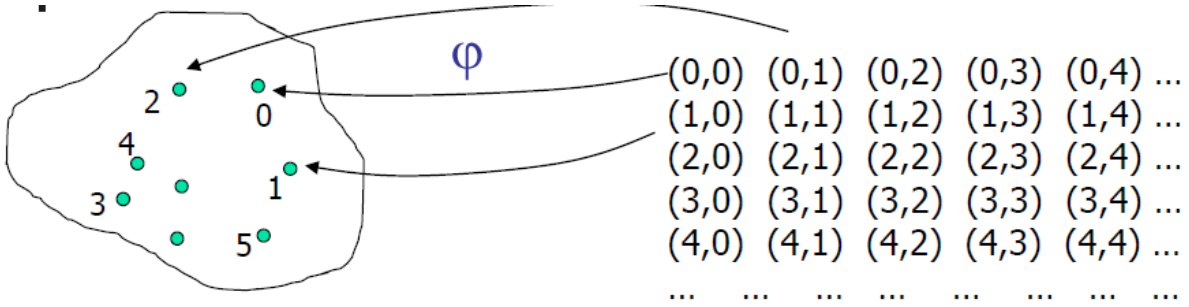


Другая нумерация кортежей натуральных чисел – метод нумерации счётного объединения счётных $\varphi(a, b) = 2^a (2b + 1) - 1;$

0	1	2	3	4	5	6	7	8	9 ...
(0,0)	(1,0)	(0,1)	(2,0)	(0,2)	(1,1)	(0,3)	(3,0)	(0,4)	(1,2) ...

Третье представление элементов множества натуральными числами

φ – функция нумерации



$$n = \varphi(a, b) = (a+b)^2 + a$$

a \ b	0	1	2	3	4	
0	0	1	4	9	16	...
1	2	5	10	17	26	...
2	6	11	18	27	38	...
3	12	19	28	39	52	...
4	20	29	40	53	68	...
...	...	...	...	...	...	...

Геделева нумерация

- Геделева нумерация – это немного другой способ упорядочения всех строк над конечным словарем - приписывание каждой строке α ее "геделева номера" $\gamma(\alpha)$, натурального числа – отображения

$$\gamma: \Sigma^* \rightarrow \mathbf{N}$$

- Любой формуле или последовательности формул, образующих доказательство, Гедель ставил в соответствие натуральное число - геделевский номер.
- Требование к такой нумерации:
  а) любой строке символов α из Σ приписывается единственное натуральное число $\gamma(\alpha)$
  б) отображение γ должно позволять однозначно восстановить всю строку, т.е. все входящие в него символы-компоненты и их позиции. Иными словами, должно быть однозначным обратное отображение

$$\gamma^{-1}: \mathbf{N} \rightarrow \Sigma^*$$

- Как построить такое отображение? Например, пусть словарь (алфавит) Σ содержит символы {+ , x, =, 0, ', (,)}. Строки символов в этом словаре:
  x = x
  x+x′ = x′+x
) = x + + 0 ( ) - синтаксически неправильная строка. Но тоже строка!

Геделева нумерация

- Общий вид символьной строки в алфавите: $a_1\ a_2\ a_3\ a_4\ a_5\ \ldots$, т.е. строка определяется номерами позиций в ней и теми символами, которые стоят в каждой позиции, начиная с 0
  Пример: словарь Σ содержит символы {+ , x, =, 0, ‘, (,)}
  строка α: x = x
  в 1-й позиции α – первый символ x словаря Σ
  во 2-й позиции позиции α – второй символ = словаря Σ
  в 3-й позиции α – первый символ x словаря Σ
- Итак, нужно приписать цепочке символов α такой единственный номер, чтобы он однозначно определялся набором пар
  <номер позиции, номер символа словаря в этой позиции>
  Строка α: x = x должна представляться номером $\gamma(\alpha)$, однозначно определяемом тремя парами:
  <1, 1>, <2,3>, <3,1>
  но так, чтобы эти три пары однозначно восстанавливались по числу
- Гедель поставил вопрос: Как можно представить любое натуральное число взаимно однозначно набором пар натуральных чисел?

Геделева нумерация

- Известно, что любое натуральное число N может быть однозначно представлено разложением по своим простым делителям, не большим N/2:

  $N=2^{n1} \times 3^{n2} \times 5^{n3} \times 7^{n4} \times \ldots . =$

  1 2 3 4 ... = это номера простых чисел pi

- Например,

  $20 = 4 \times 5 = 2^2 \times 3^0 \times 5^1$

  $90 = 10 \times 9 = 2 \times 3 \times 3 \times 5 = 2^1 \times 3^2 \times 5^1$

- Итак, каждое разложение N однозначно определяется набором пар:

  <номер простого числа, степень этого числа в разложении N>

  Пример:

  $20= 2^2 \times 5^1 \Rightarrow$<1,2>, <2,0>, <3,1>

  $90 = 2^1 \times 3^2 \times 5^1 \Rightarrow$<1,2>, <2,2>, <3,1>

- Каждая цепочка конечного словаря однозначно определяется набором пар

  <номер позиции, номер символа словаря в этой позиции>

Геделевская нумерация строк словаря

- Символам словаря $\Sigma = \{+ , x, =, 0, `, (,)\}$:
  сопоставляются нечетные числа, большие 1
- Позиции j символа в формуле сопоставляется j-е простое число

Упорядочиваем словарь – i- му символу словаря – i-е натуральное число:

$\Sigma = \{ \; +, \quad x, \quad =, \quad 0, \quad `, \quad (, \quad), \quad * \; \}$

номера символов: 1 2 3 4 5 6 7 8

цепочке **(x * x) = 0 ’** сопоставляется геделев номер

N символов: 6 2 8 2 7 3 4 5

N позиции: **2 3 5 7 11 13 17 19** - последовательность простых

геделев номер цепочки:

$$2^6 \times 3^2 \times 5^8 \times 7^2 \times 11^7 \times 13^3 \times 17^4 \times 19^5 = N$$

Каждое натуральное число ОДНОЗНАЧНО раскладывается в произведение степеней его простых множителей. Следовательно, восстановление исходной цепочки по ее номеру – однозначно

После разложения **N** на простые множители получим:

На 1-й позиции формулы – символ (, на 2-й позиции – символ x, ...

Числовые функции

- Числовые функции – аргументы и результаты которых – натуральные числа (включая 0)
- Числа всегда представлены в каком-либо алфавите ю Примеры:
  - Двоичная запись в алфавите {0,1}
  - Десятичная запись в алфавите {0, 1, 2, …, 9}
  - Унарная запись в алфавите {|}
- Всякая числовая функция вычислимая при одной форме записи, вычислима и в другой (не зависит от способа записи). Пусть C и D – алгоритмы перехода от второй формы записи к первой, а D - обратно, перевод из первой формы во вторую. Тогда алгоритм CAD(x) вычисляет f(x) во второй форме записи, если A(x) – алгоритм, вычисляющий f(x) в первой форме



Три типа универсальных алгоритмических моделей

Первый тип: алгоритмы вида $F: N \to N$

числовые функции на множестве натуральных чисел

Наиболее изученная модель этого типа - рекурсивные функции (Алонзо Черч) - первая формализация понятия алгоритма. Берется набор из трех простейших базовых функций и несколько операции над ними

Второй тип: алгоритмы преобразования цепочек вида $\Phi: \Sigma^* \to \Sigma^*$

пошаговые преобразования слов в произвольных алфавитах

Элементарные операции – подстановки $\alpha \to \beta$, т.е. замены части α входного слова другим словом β.

Примеры этого типа - нормальные алгоритмы Маркова, системы Поста

Третий тип: алгоритмы преобразования цепочек вида $\Phi: \Sigma^* \to \Sigma^*$

представление алгоритма, как автомата над множеством входных цепочек

автомат - устройство, читающее по шагам символы входного слова, и выдающее на выход символы выходного слова в зависимости от его состояния и входного символа.

Примеры этого типа – конечный автомат, машина Тьюринга, машина Поста



Представление алгоритмов рекурсивными арифметическими функциями

- Числовые функции F:N->N, значение которых можно установить посредством некоторого алгоритма, называются вычислимыми.
- А.Черч предложил определить все вычислимые функции как арифметические (целочисленные) функции, которые могут быть конструктивно (с помощью рекурсии) определены через простейшие. Они были названы *рекурсивными функциями.*



Рекурсивные и частично-рекурсивные функции

- *Рекурсией* называется способ задания функции, при котором значение определяемой функции для произвольных значений аргументов выражается через значения этой же функции для меньших значений аргументов.
- Применение рекурсивных функций в теории алгоритмов основано на идее нумерации слов в произвольном алфавите последовательными натуральными числами в лексикографическом порядке (располагая слова в порядке возрастания их длин, а слова, имеющие одинаковую длину, - в порядке номеров символов).
- После нумерации входных и выходных слов в произвольном алфавитном операторе этот оператор превращается в функцию $y = f(x)$, в которой аргумент x и результат функции y принимают неотрицательные целочисленные значения.
- Обычно функция может быть определена не для всех значений *аргумента,* а лишь для тех, которые составляют область определения этой функции. Такая функция называется частично-рекурсивной.

Три простейшие функции и операции композиции функций

- Для того чтобы описать класс функций с помощью рекурсивных определений, рассмотрим набор простейших функций:
  - 1) $Z(x_1, x_2, ..., x_n) = 0$ - нуль-функция, которая определена для всех неотрицательных значений аргумента;
  - 2) $s(x) = x+1$ - функция непосредственного следования, также определенная для всех целых неотрицательных значений аргумента;
  - 3) $I_m^n (x_1, x_2, . . ., x_m, . . . , x_n) = x_m$ - функция выбора. (тождества), повторяющая значения своих аргументов
- Используя простейшие функции в качестве исходных функций, можно с помощью небольшого числа общих конструктивных приемов строить сложные арифметические функции.
- В теории рекурсивных функций особо важное значение имеют три операции:
  - суперпозиции,
  - примитивной рекурсии,
  - минимизации.

Оператор суперпозиции

Оператором суперпозиции S называется подстановка в функцию от m переменных m функций от n одних и тех же переменных. Она дает новую функцию от n переменных.

- Например, из функций $f(x_1, ..., x_m)$, $f_1(x_1, ..., x_n)$, . . ., $f_m(x_1, ..., x_n)$ можно получить новую функцию:
  $S^{m+1}(f, f_1, ..., f_m) = g(x_1, ..., x_n) = f(f_1(x_1, ..., x_n), ..., f_m(x_1, ..., x_n))$

  В операции суперпозиции S^{m+1} индекс сверху указывает на число функций.

Таким образом, при помощи оператора суперпозиции и функции выбора можно выразить *любую подстановку* функции в функцию.

- Например, осуществляя операцию суперпозиции функций f(x) = 0 и g(x) = x+1, получим функцию: h(x) = g(f(x)) = 0 + 1 = 1. При суперпозиции функции g(x) с этой же функцией получим функцию h(x) = g(g(x)) = x + 2.
- Используя подстановку и функции тождества, можно переставлять и отождествлять аргументы в функции:

  f(x2, x1, x3, . . ., xn) = f(I22(x1, x2), I12(x1, x2), x3, . . ., xn);

  f(x1, x1, x3, . . ., xn) = f(I12(x1, x2), I12(x1, x2), x3, . . ., xn).

Таким образом, если заданы функции тождества и операторы суперпозиции, то можно считать заданными всевозможные операторы подстановки функций в функции, а также переименования, перестановки и отождествления переменных.



Оператор примитивной рекурсии

- Этот оператор R_n позволяет определить (n+1) - местную функцию f по двум заданным функциям, одна из которых является n- местной функцией g, а другая - (n+2) - местной функцией h.

  $g(x_1, ..., x_n)$ – n-местная

  $h(x_1, ..., x_n, y, z)$ - n+2-местная

- Функция $f(x_1, ..., x_n, y)$ получается оператором примитивной рекурсии из функций $g(x_1, ..., x_n)$ и функции $h(x_1, ..., x_n, y, z)$, если:

  $f(x_1, ..., x_n, 0) = g(x_1, ..., x_n);$ (*)

  $f(x_1, ..., x_n, y+1) = h(x_1, ..., x_n, y, f(x_1, ..., x_n, y))$.

- Приведенная пара равенств называется ***схемой примитивной рекурсии***.
- Для понимания операции примитивной рекурсии необходимо заметить, что всякую функцию от меньшего числа аргументов можно рассматривать как функцию от большего числа аргументов. Существенным в операторе примитивной рекурсии является то, что независимо от числа переменных в f рекурсия ведется только по одной переменной y. Остальные n переменных $x_1, x_2, ..., x_n$ на момент применения схемы (*) зафиксированы и играют роль параметров.

Оператор минимизации

- Отношение $P(x_1, ..., x_n)$ называется *примитивно-рекурсивным*, если примитивно-рекурсивна его характеристическая функция:

$$\chi = \begin{cases} 0, \text{ если } P(x_1, ..., x_n) \text{ - " ложь”} \\ 1, \text{ если } P(x_1, ..., x_n) \text{ - "истина" ,} \end{cases}$$

  Предикат называется примитивно-рекурсивным, если его характеристическая функция примитивно-рекурсивна.

- Функция $f(x_1, ..., x_n)$ получается посредством оператора минимизации из предиката $P(x_1, ..., x_n, z)$, если в любой точке значением функции $f(x_1, ..., x_n)$ является минимальное значение z, обращающее предикат $P(x_1, ..., x_n, z)$ в значение «истина»:

$$f(x_1, ..., x_n) = \mu_z (P(x_1,..., x_n, z)),$$

  где μ_z – оператор минимизации.

Ограниченный оператор минимизации

- Функция $f(x_1, ..., x_n)$ получается ограниченным оператором минимизации из предиката $P(x_1,..., x_n, z)$ и функции $U(x_1, ..., x_n)$, если в любой точке значение этой функции определено следующим образом:

  1) для любого $z < U(x_1, ..., x_n)$ такого, что $P(x_1, ..., x_n, z)$ = "истина",
  значение функции $f(x_1, ..., x_n) = \mu_z(P(x_1, ..., x_n, z))$

  2) не для любого $z < U(x_1, ..., x_n)$ такого, что $P(x_1, ..., x_n, z)$ = "истина", значение функции $f(x_1, ..., x_n) = U(x_1, ..., x_n)$.

- Данное определение записывается следующим образом:
  $f(x_1, ..., x_n) = \mu_{z < U(x)}(P(x_1, ..., x_n, z))$

Ограничение z в ограниченном операторе минимизации дает гарантию окончания вычислений, поскольку оно оценивает сверху число вычислений предиката P. Возможность оценить сверху *количество вычислений* является существенной особенностью примитивно-рекурсивных функций.

Примитивно-рекурсивные функции

- Любые рекурсивные функции вычислимы с помощью определенных процедур механического характера из простейших.

- Функция называется *примитивно-рекурсивной*, если она может быть получена из простейших функций с помощью конечного числа операторов суперпозиции и примитивной рекурсии
- Если некоторые функции являются примитивно-рекурсивными, то в результате применения к ним операторов суперпозиции или примитивной рекурсии можно получить новые примитивно-рекурсивные функции.
- Существует три возможности доказательства того, что функция является примитивно-рекурсивной:
  - а) показать, что заданная функция является простейшей;
  - б) показать, что заданная функция построена с помощью оператора суперпозиции;
  - в) показать, что заданная функция построена с помощью оператора примитивной рекурсии.



Частично-рекурсивные функции

- Функция называется *частично-рекурсивной*, если она может быть получена из простейших функций с помощью следующих операторов, выполненных в любой последовательности и любое конечное число раз
  - суперпозиции
  - примитивной рекурсии
  - минимизации.
- Если такие функции всюду определенные, они называются общерекурсивными. Если не всюду определенные, то они называются *частично-рекурсивными*.

Тезис Черча: ВСЯКИЙ АЛГОРИТМ МОЖЕТ БЫТЬ РЕАЛИЗОВАН ЧАСТИЧНО-РЕКУРСИВНОЙ ФУНКЦИЕЙ.

- Факт: Все известные до сих пор алгоритмы могут быть представлены частично-рекурсивными функциями.
- **Утверждение о несуществовании частично-рекурсивной функции эквивалентно несуществованию алгоритма решения соответствующей задачи.**

Модели вычислений

- Модели вычисления начали возникать задолго до появления первых языков программирования, однако впоследствии открылось неожиданное их применение. Стало возможным использовать теоретические результаты, полученные для различных моделей вычисления, для ***исследования собственно языков программирования***.
- Далеко не всегда это было ясно создателям языков программирования.
- Понятие **λ-определимых функций** и **λ-исчисление** лежат в основе *функционального программирования*, идеи и подходы которого применяются сейчас практически во всех достаточно распространённых современных языках программирования.
- Одной из важнейших характеристик функционального программирования является использование функций как значений первого класса («first class value»). Это означает, что функции наряду с данными других типов — числами, логическими значениями, списками — могут быть *параметрами или возвращаемыми значениями* других функций, а кроме того с ними можно выполнять различные операции, например, композицию (суперпозицию).
- Такое широкое применение функций требует введения удобной нотации.

Пример

Попробуем определить **смысл выражения** $x - y$. $f(x) = x - y,$

Во-первых, его можно трактовать как функцию переменной x:

$$f : x \mapsto x - y$$

С другой стороны то же самое выражение можно понимать как функцию переменной y:

$$g(y) = x - y \qquad g : y \mapsto x - y.$$

Поскольку в этих примерах смысл выражения с точки зрения функции меняется кардинальным образом, необходим способ, позволяющий **явно** указать, что именно является аргументом функции, но при этом желательно *избежать её именования*.

Подстановки и замены

Пусть T — множество термов над множеством переменных V и сигнатурой Σ. *Подстановкой*, или *заменой переменных*, называется множество пар

$$\sigma = \{t_i//v_i\}_{i \in I},$$

где t_i — термы, а v_i — переменные. Результатом применения подстановки σ к терму t (обозначается $t\sigma$) называется терм, который получается заменой *всех* вхождений переменных v_i соответствующими термами t_i. *Композицией* подстановок $\sigma_1 = \{t_i//v_i\}_{i \in I}$ и $\sigma_2 = \{t_j//v_j\}_{j \in J}$ называется подстановка $\sigma_1 \circ \sigma_2 \stackrel{\text{Def}}{=} \{t_k//v_k\}_{k \in K}$, где $K = I \cup J$, а

$$t_k = \begin{cases} t_i\sigma_2, & \text{если } k \in I, \\ t_j, & \text{если } k \notin I. \end{cases}$$

Множество подстановок образует моноид относительно композиции, причём тождественная подстановка $\{v_i//v_i\}$ является единицей.

1. Замена всех вхождений переменной: $x \vee \neg x\{y \wedge z//x\} = (y \wedge z) \vee \neg(y \wedge z)$.
2. Замена всех вхождений подформулы: $x \vee y \vee z\{\neg x//y \vee z\} = x \vee \neg x$.
3. Замена первого вхождения переменной: $x \vee \neg x\{y/x\} = y \vee \neg x$.
4. Замена первого вхождения подформулы: $x \vee y \vee z\{\neg x/y \vee z\} = x \vee \neg x$.

λ-ИСЧИСЛЕНИЕ

- Соответствующая нотация была изобретена в 30-е годы XX века американским логиком и математиком Алонсо Чёрчем (1903—1995). В этой нотации в качестве вспомогательного символа используется греческая буква λ.
- Примеры в λ-нотации Чёрча выглядят так:

$$f = \lambda x\,.\,x - y,\ \ g = \lambda y\,.\,x - y$$

$$\begin{aligned}(\lambda x\,.\,x - y)(42) &= 42 - y\\ (\lambda y\,.\,x - y)(42) &= x - 42\end{aligned}$$

λ-ИСЧИСЛЕНИЕ

- Если необходимо использовать функции многих переменных, λ-нотацию нетрудно расширить, записав, например:

$$h(x, y) = x - y = \lambda xy \,.\, x - y.$$

- Однако этого расширения можно избежать, заменив функции многих переменных на функции одной переменной, значениями которых являются другие функции.

$$h^* = \lambda x \,.\, (\lambda y \,.\, x - y)$$

- Значением функции h* в точке a будет уже другая функция:

$$h^*(a) = (\lambda x \,.\, (\lambda y \,.\, x - y))(a) = \lambda y \,.\, a - y.$$

- Вычислим теперь (h*(a))(b):

$$(h^*(a))(b) = (\lambda y \,.\, a - y)(b) = a - b = h(a, b).$$

- Таким образом, можно считать, что функция h* представляет ту же функцию, что и h, но является функцией одной переменной. Такой способ записи функций многих переменных называется ***каррированием*** по имени другого американского логика Хаскелла Карри (1900—1982). Каррирование полезно еще и по той причине, что оно явно вводит в рассмотрение *функции высшего порядка* (higher-order functions), т.е. функции, аргументами или значениями которых среди прочего могут являться функции.

Индуктивное определение множества λ-термов

- Введённая Чёрчем λ-нотация важна не сама по себе, а как инструмент построения *формальной системы λ-исчисления*, которое является одной из ***формализаций понятия алгоритма*** и одновременно ***теоретической основой функционального программирования***.
- Идея простая: перечисляем простейшие элементы (**термы**) множества (базис индукции), а потом указываем **правила для построения** из них всех остальных (индуктивный переход).

- Пусть задано счётное множество переменных x_1, x_2, x_3, … Множество выражений, называемых ***λ-термами***, определяется с помощью следующих трёх условий:

  - Все переменные являются λ-термами.
  - Если M и N — произвольные λ-термы, то (MN) — λ-терм, называемый применением (application).
  - Если x — переменная, а M — произвольный λ-терм, то $(\lambda x\,.\,M)$ — λ-терм, называемый абстракцией (abstraction).

Индуктивное определение множества λ-термов

- Введённым понятиям можно придать следующую неформальную интерпретацию. Так, можно считать, что λ-абстракция обозначает функцию одной переменной, причём переменная, указанная после символа λ, является её формальным параметром.
- При таком подходе применение MN означает вызов функции M с передачей ей в качестве фактического параметра выражения N. То, что при этом используется бесскобочная запись (в отличие от более традиционного $f(x)$), может напоминать, например, о выражениях с тригонометрическими функциями:

$$\operatorname{tg} x = \frac{\sin x}{\cos x}$$

- В качестве переменных далее будем использовать буквы x, y, z, s, t, u, v или w с индексами и без, а прописными латинскими буквами M, N, P и Q будем обозначать произвольные λ-термы.

Примеры λ-термов

1. x
2. (xy)
3. $(\lambda x\,.\,x)$
4. $((\lambda x\,.\,y)\,t)$
5. $(\lambda x\,.\,y)$
6. $(\lambda x\,.\,(\lambda y\,.\,(xy)))$
7. $((\lambda x\,.\,(\lambda x\,.\,(xu)))(\lambda x\,.\,v))$

- Терм (1) – это просто переменная, терм (2) представляет собой применение переменной x к переменной y (в определении не требуется, чтобы слева в применении стояла функция), терм (3) можно трактовать как тождественную функцию, терм (5) — как константную, терм (7) — это применение одной абстракции к другой;
- Здесь можно обратить внимание на многократное использование переменной x в λ-абстракциях, определение этому не препятствует, хотя при понимании таких выражений возможны определённые трудности.
- *Символ λ и выражение λx . термами не являются, они используются только как фрагменты λ-абстракции.*

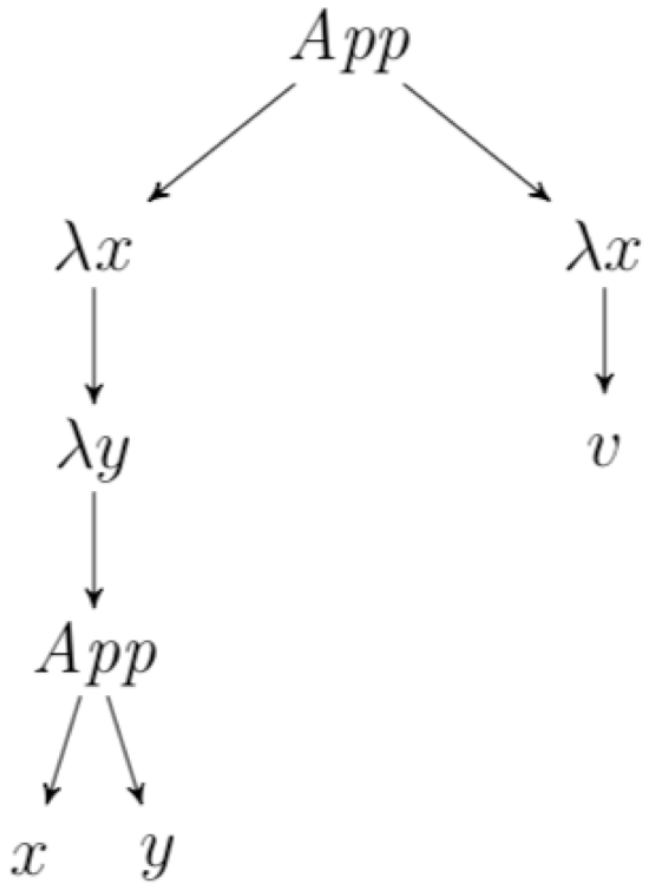
Упрощения

- Упрощая запись термов, примем соглашение о том, что в выражениях с λ-термами можно опускать все внешние скобки, а также некоторые внутренние, считая, что:
  - применение является левоассоциативным, то есть выражение MNP понимается как $((MN)P)$;
  - абстракция является правоассоциативной, причём символ λ захватывает максимально большое выражение, то есть терм $\lambda x\,.\,MN$ соответствует $(\lambda x\,.\,(MN))$, а терм $\lambda x\,.\,\lambda y\,.\,\lambda z\,.\,M$ означает $(\lambda x\,.\,(\lambda y\,.\,(\lambda z\,.\,M)))$.

x	x
(xy)	xy
$(\lambda x\,.\,x)$	$\lambda x\,.\,x$
$((\lambda x\,.\,y)\,t)$	$(\lambda x\,.\,y)\,t$
$(\lambda x\,.\,y)$	$\lambda x\,.\,y$
$(\lambda x\,.\,(\lambda y\,.\,(xy)))$	$\lambda x\,.\,\lambda y\,.\,xy$
$((\lambda x\,.\,(\lambda x\,.\,(xu)))(\lambda x\,.\,v))$	$(\lambda x\,.\,\lambda x\,.\,xu)\,(\lambda x\,.\,v)$

Дерево разбора терма

- Скобки отражают структуру терма, которая на самом деле определяется последовательностью использования конкретных правил из определения при его построении.
- Например, терм $(\lambda x\ .\ \lambda y\ .\ xy)\ (\lambda x\ .\ v)$ — это применение из двух абстракций, тело первой из которых есть снова абстракция с телом — применением одной переменной к другой, а тело второй есть переменная. Это крайне неуклюжее описание гораздо удобнее представить графически, нарисовав дерево разбора терма.
- Узлы дерева вида App обозначают применение, узлы λx и λy — абстракции по соответствующим переменным, всё остальное — узлы переменных.



Свободные и связанные переменные

Областью действия (scope) λ-абстракции λx в терме $(\lambda x.M)$ называется терм M. ***Вхождением переменной*** в терм называется любое её использование внутри этого терма (в том числе и сразу после символа λ).

- Вхождение переменной x в терм M называется *связанным* (bound), если оно находится в области действия некоторой абстракции λx в терме M или является следующей за символом λ буквой x;
- *свободным* (free) во всех остальных случаях.

В этом определении идёт речь о связанных и свободных вхождениях переменной x, а не о самой переменной x. Переменная может входить в терм несколько раз, причём некоторые из её вхождений могут оказаться свободными, а другие — связанными.

- Например в терме $x\lambda x.x$ имеется три вхождения переменной x: первое вхождение является свободным, второе и третье — связанными.

Множеством свободных переменных терма P (обозначается $FV(P)$ от англ. free variables), будем называть множество переменных, имеющих в терме P хотя бы одно свободное вхождение:

- Структура определения повторяет структуру определения множества λ-термов.

$$\begin{aligned} FV(x) &= \{x\} \\ FV(\lambda x.M) &= FV(M)\backslash\{x\} \\ FV(MN) &= FV(M) \cup FV(N) \end{aligned}$$

Свободные и связанные переменные

- Множеством связанных переменных терма P (обозначается BV (P) от англ. bound variables), будем называть множество переменных, имеющих в терме P хотя бы одно связанное вхождение:

$$\begin{aligned} BV(x) &= \varnothing \\ BV(\lambda x.M) &= \{x\} \cup BV(M) \\ BV(MN) &= BV(M) \cup BV(N) \end{aligned}$$

- В соответствии с этими определениями множества свободных и связанных переменных терма $x\ \lambda x\ .\ x$ будут совпадать:

$$\begin{aligned} FV(x\,\lambda x\,.\,x) &= FV(x) \cup FV(\lambda x\,.\,x) = \{x\} \cup (FV(x) \backslash \{x\}) = \\ &= \{x\} \cup (\{x\} \backslash \{x\}) = \{x\} \cup \varnothing = \{x\}; \\ BV(x\,\lambda x\,.\,x) &= BV(x) \cup BV(\lambda x\,.\,x) = \varnothing \cup (\{x\} \cup BV(x)) = \\ &= \varnothing \cup (\{x\} \cup \varnothing) = \{x\}. \end{aligned}$$

Подстановка. Неправильное определение

Операцией подстановки терма N (фактический параметр) в терм P (подпрограмма) вместо переменной x (формальный параметр), обозначаемой как $[N/x]P$, называется операция преобразования терма, выполняемая по правилам:

$$
\begin{array}{lcl}
[N/x]x & = & (N); \\
[N/x]y & = & y, \quad \textit{если } x \neq y; \\
[N/x](PQ) & = & (([N/x]P)([N/x]Q)); \\
[N/x](\lambda x\,.\,P) & = & (\lambda x\,.\,([N/x]P)); \\
[N/x](\lambda y\,.\,P) & = & (\lambda y\,.\,([N/x]P)), \quad \textit{если } x \neq y.
\end{array}
$$

Проблемы этого определения скрываются в двух последних правилах.

- **При выполнении подстановки не должен меняться смысл терма.**
- Посмотрим, что произойдёт с термом $\lambda x.x$ при проведении подстановки $[t/x]$ по сформулированным выше правилам:

$$[t/x](\lambda x\,.\,x) = \lambda x\,.\,([t/x]x) = \lambda x\,.\,t.$$

- Получается, что *тождественная функция превратилась в константную*, а связанная в исходном терме переменная x исчезла, превратившись в свободную переменную t.

Подстановка. Неправильное определение

Операцией подстановки терма N (фактический параметр) в терм P (подпрограмма) вместо переменной x (формальный параметр), обозначаемой как $[N/x]P$, называется операция преобразования терма, выполняемая по правилам:

$$
\begin{array}{lcl}
[N/x]x & = & (N); \\
[N/x]y & = & y, \quad \textit{если } x \neq y; \\
[N/x](PQ) & = & (([N/x]P)([N/x]Q)); \\
[N/x](\lambda x\,.\,P) & = & (\lambda x\,.\,([N/x]P)); \\
[N/x](\lambda y\,.\,P) & = & (\lambda y\,.\,([N/x]P)), \quad \textit{если } x \neq y.
\end{array}
$$

Проблемы этого определения скрываются в двух последних правилах.

- **При выполнении подстановки не должен меняться смысл терма.**
- Выполним подстановку $[y/x]$ в терме $\lambda y\,.\,x$:

$$[y/x](\lambda y\,.\,x) = \lambda y\,.\,([y/x]x) = \lambda y\,.\,y.$$

- Произошло нечто противоположное: константная функция превратилась в тождественную. В таких случаях обычно говорят, что произошёл *захват свободной переменной* y, имея ввиду, что в подставляемом терме y выступала в качестве свободной переменной, а после подстановки она же оказалась связанной.



Подстановка. Неправильное определение

- Таким образом, корректное определение подстановки должно учитывать связанные и свободные переменные, препятствуя исчезновению одних и захвату других.
- Проблему ***с исчезновением связанной переменной*** можно решить с помощью запрета на распространение подстановки $[N/x]$ внутрь абстракции $\lambda x\ .\ P$, причём это вполне согласуется с пониманием смысла формального параметра подпрограммы: он никак не зависит от действий во внешних по отношению к данной подпрограмме частях программы.
- Проблема ***с захватом свободных переменных*** может быть решена с помощью переименования связывающих переменных в соответствующих λ-абстракциях.
- Это решение не противоречит введенной интерпретации, поскольку переименование формального параметра подпрограммы с одновременной заменой имени параметра в её теле не приводит к изменению смысла ни данной подпрограммы, ни программы в целом.



Подстановка. Правильное определение

- ***Операцией подстановки терма*** N в терм P вместо переменной x, обозначаемой как $[N/x]P$, называется операция преобразования терма, выполняемая по правилам:

$$
\begin{array}{lcll}
[N/x]x & = & (N) & \\
[N/x]y & = & y, & \text{если } x \neq y \\
[N/x](PQ) & = & (([N/x]P)([N/x]Q)) & \\
[N/x](\lambda x\,.\,P) & = & (\lambda x\,.\,P) & \\
[N/x](\lambda y\,.\,P) & = & (\lambda y\,.\,P), & \text{если } x \notin FV(P) \\
[N/x](\lambda y\,.\,P) & = & (\lambda y\,.\,([N/x]P)), & \text{если } x \in FV(P) \text{ и } y \notin FV(N) \\
[N/x](\lambda y\,.\,P) & = & (\lambda z\,.\,([N/x]([z/y]P))), & \text{если } x \in FV(P) \text{ и } y \in FV(N), \\
 & & & \text{причём } z \notin FV(NP)
\end{array}
$$

- Вместо двух последних правил неправильного определения в правильном появились *четыре новых правила*, первое из которых решает проблему исчезновения связанных переменных, а три последних — проблему захвата свободных. Протестируем новое определение на старых примерах (с точностью до имени связывающей переменной):

$$
\begin{array}{l}
[t/x](\lambda x\,.\,x) = \lambda x\,.\,x, \\
[y/x](\lambda y\,.\,x) = \lambda z\,.\,([y/x]([z/y]x)) = \lambda z\,.\,([y/x]x) = \lambda z\,.\,y.
\end{array}
$$

Эквивалентность

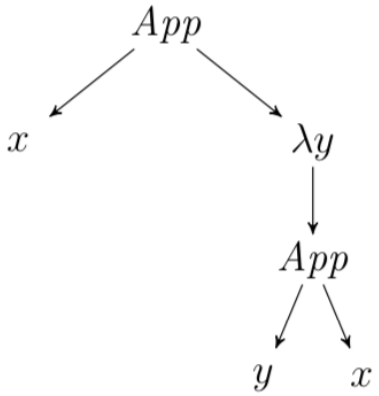
- Замена терма $\lambda x.M$ на терм $\lambda y.[y/x]M$ (то есть переименование связанной переменной) называется ***α-конверсией***. Два терма P и Q, один из которых можно получить из другого с помощью конечного числа α-конверсий, называются ***α-эквивалентными***, что обозначается как $P \equiv_\alpha Q$.

- Например, следующие термы α-эквивалентны:

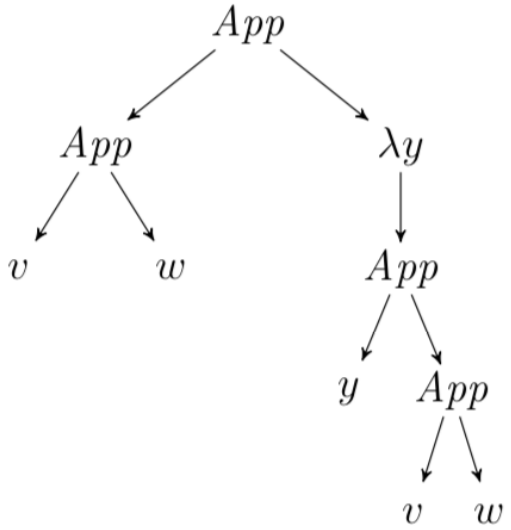
$$\begin{aligned} \lambda x\,.\,x &\equiv_\alpha \lambda y\,.\,y; \\ y\,\lambda y\,.\,y &\equiv_\alpha y\,\lambda z\,.\,z; \\ \lambda x\,.\,(\lambda y\,.\,x) &\equiv_\alpha \lambda y\,.\,(\lambda x\,.\,y) \end{aligned}$$

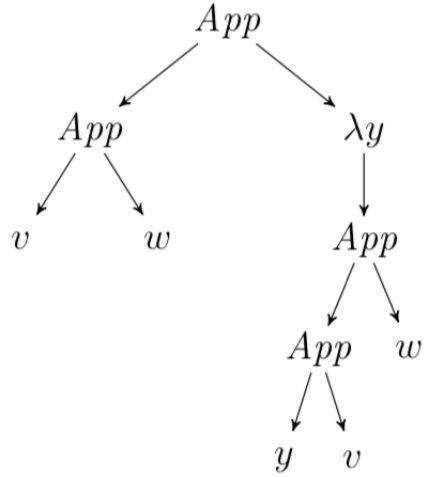
Пример 1

В результирующем выражении вокруг второго вхождения vw появились скобки. Они не требовались, когда в соответствующей позиции находилась переменная x, но стали нужны теперь, когда её заменили на применение. Подстановка не должна нарушать структуру терма: поддерево дерева разбора, соответствующее заменяемой переменной, изменяется, но всё, что находится вне его, изменяться не должно.



до

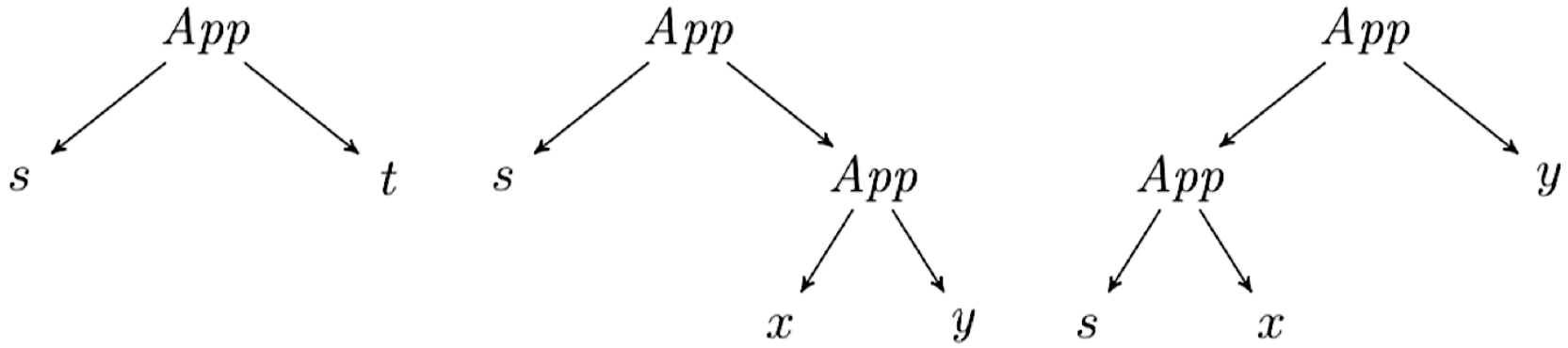




без скобок (ошибочно)

Пример 2

$$[xy/t]st = s(xy) \neq sxy = (sx)y$$



исходное **после подстановки** **без скобок**

Редукция термов

Термы вида $(\lambda x.M)N$ называются ***редексами*** или ***редуцируемыми выражениями*** (reducible expression).

- Если считать λ-абстракцию функцией, а применение — вызовом функции, то редекс представляет собой вызов функции, явно заданной абстракцией. Выполнив такой вызов, то есть подставив фактический параметр на место формального, можно получить результат вызова:

$$[N/x]M.$$

- Отношение, которое ставит в соответствие редексу $(\lambda x.M)N$ терм $[N/x]M$ называется ***β-редукцией***. Если терм P содержит в качестве своего фрагмента редекс $(\lambda x.M)N$, который сводится (редуцируется) к терму $[N/x]M$, причём после замены редекса на этот терм получается терм P', то будем писать, что

$$P \to_\beta P'.$$

- Если терм P' можно получить из терма P с помощью конечного числа шагов β-редукции, будем обозначать этот факт как

$$P \to_\beta^* P'.$$

Редукция термов

1. $(\lambda x\,.\,x)\,y \to_\beta y$;
2. $(\lambda x\,.\,y)\,t \to_\beta y$;
3. $(\lambda x\,.\,x(xy))N \to_\beta N(Ny)$.

Говорят, что λ-терм находится в нормальной форме, если он не содержит ни одного редекса. Для приведения терма в нормальную форму, вообще говоря, может потребоваться несколько шагов редукции:

$$
\begin{aligned}
(\lambda x\,.\,\lambda y\,.\,yx)\,z\,v &\to_\beta ([z/x](\lambda y\,.\,yx))\,v = (\lambda y\,.\,yz)\,v \\
&\to_\beta [v/y](yz) = vz; \\
(\lambda x\,.\,\lambda y\,.\,x)(\lambda z\,.\,z)\,t &\to_\beta ([\lambda z\,.\,z/x](\lambda y\,.\,x))\,t = (\lambda y\,.\,\lambda z\,.\,z)\,t \\
&\to_\beta [t/y](\lambda z\,.\,z) = \lambda z\,.\,z.
\end{aligned}
$$

Представление данных в λ-исчислении

- λ-исчисление можно рассматривать как простой язык программирования. Далее нашей задачей будет кодирование базовых элементов любого языка программирования в терминах λ-исчисления. В числе таких элементов будут логические значения и операции, натуральные числа, пары значений, условные операции и рекурсивные вызовы.
- Введём логические значения $true$ и $false$:

$$true = \lambda x\,.\,\lambda y\,.\,x; \quad false = \lambda x\,.\,\lambda y\,.\,y.$$

- Смысл знака = в этих выражениях: договариваемся обозначать термы, стоящие в правой части равенства, символами из левой части.
- Вообще говоря, назвать истиной и ложью можно любые термы, но такой выбор обозначений оказался наиболее удобным. Истинное значение можно трактовать как функцию двух аргументов, возвращающую первый из них, а ложное значение — как возвращающую второй.

Представление данных в λ-исчислении

Используя эти определения, можно ввести условное выражение:

$$\text{if } C \text{ then } E_1 \text{ else } E_2 = C E_1 E_2$$

$$\text{if } true \text{ then } E_1 \text{ else } E_2 = true\ E_1 E_2 = (\lambda x\,.\,\lambda y\,.\,x) E_1 E_2 = E_1.$$

$$\text{if } false \text{ then } E_1 \text{ else } E_2 = false\ E_1 E_2 = (\lambda x\,.\,\lambda y\,.\,y) E_1 E_2 = E_2.$$

Пользуясь условной операцией, нетрудно определить отрицание, конъюнкцию и дизъюнкцию:

$$\begin{array}{lcll} \text{not } P & = & \text{if } P \text{ then } false \text{ else } true & = & P\ false\ true; \\ P \text{ and } Q & = & \text{if } P \text{ then } Q \text{ else } false & = & P\ Q\ false; \\ P \text{ or } Q & = & \text{if } P \text{ then } true \text{ else } Q & = & P\ true\ Q. \end{array}$$



Представление данных в λ-исчислении

Данные могут объединяться в структуры данных, простейшей такой структурой является пара:

$$(E_1, E_2) = \lambda z \,.\, z\ E_1\ E_2.$$

Для работы с парами необходимы функции доступа к отдельным компонентам:

$$\begin{aligned} \text{fst}\ P &= P\ true; \\ \text{snd}\ P &= P\ false. \end{aligned}$$

Убедимся в работоспособности этих функций:

$$\begin{aligned} \text{fst}\ (E_1, E_2) &= (\lambda z \,.\, z\ E_1\ E_2)\, true = true\ E_1\ E_2 = E_1; \\ \text{snd}\ (E_1, E_2) &= (\lambda z \,.\, z\ E_1\ E_2)\, false = false\ E_1\ E_2 = E_2. \end{aligned}$$

Натуральные числа можно ввести так:

$$\begin{aligned} 0 &= \lambda x \,.\, \lambda y \,.\, y \\ 1 &= \lambda x \,.\, \lambda y \,.\, xy \\ 2 &= \lambda x \,.\, \lambda y \,.\, x(xy) \\ 3 &= \lambda x \,.\, \lambda y \,.\, x(x(xy)) \\ &\dots \end{aligned}$$

Натуральные числа

- Натуральные числа, представленные таким образом, называют ***нумералами Чёрча***. Нумерал Чёрча можно интерпретировать как *функцию двух переменных*, которая применяет свой первый аргумент к второму соответствующее число раз, например:

$$\begin{aligned} 2\ f\ t &= (\lambda x\,.\,\lambda y\,.\,x(xy))\ f\ t \\ &= (\lambda y\,.\,f(fy))\ t \\ &= f(ft). \end{aligned}$$

- Можно ввести *функцию следования*, которая ставит в соответствие каждому натуральному числу следующее за ним ($n \mapsto\!\!\rightarrow n + 1$):

$$\mathrm{succ} = \lambda n\,.\,\lambda x\,.\,\lambda y\,.\,x(nxy)$$

Получение натуральных чисел

Вычислим число, следующее за числом 3:

$$\begin{aligned}
\text{succ } 3 &= (\lambda n\,.\,\lambda x\,.\,\lambda y\,.\,x(nxy))(\lambda x\,.\,\lambda y\,.\,x(x(xy))) \\
&= \lambda x\,.\,\lambda y\,.\,x\,((\lambda x\,.\,\lambda y\,.\,x(x(xy)))xy) \\
&= \lambda x\,.\,\lambda y\,.\,x\,((\lambda y\,.\,x(x(xy)))y) \\
&= \lambda x\,.\,\lambda y\,.\,x\,(x(x(xy))) \\
&= 4.
\end{aligned}$$

- Такое определение соответствует нашей интерпретации: терм, находящийся в скобках ($\boldsymbol{nxy}$) означает n-кратное применение x к y, после этого следует еще одно применение x к полученному ранее результату.
- В итоге получается, что x применяется к y в точности n + 1 раз. В целом, процесс вычислений можно трактовать как (n+1)-кратное прибавление единицы к нулю. Пользуясь той же идеей, можно сформулировать и другое определение функции следования, прибавляя n раз единицу к единице:

$$\text{succ}' = \lambda n\,.\,\lambda x\,.\,\lambda y\,.\,nx(xy).$$

Логические переменные

Связь между натуральными числами и логическими значениями можно организовать, определив функцию, проверяющую, является ли заданное число нулём:

$$\text{is_zero?} = \lambda n\,.\,n\,(\lambda x\,.\,false)\,true.$$

$$\begin{aligned}
\text{is_zero? } 0 &= (\lambda n\,.\,n\,(\lambda x\,.\,false)\,true)\,(\lambda x\,.\,\lambda y\,.\,y) \\
&= (\lambda x\,.\,\lambda y\,.\,y)(\lambda x\,.\,false)\,true \\
&= (\lambda y\,.\,y)\,true \\
&= true; \\
\text{is_zero? } 1 &= (\lambda n\,.\,n\,(\lambda x\,.\,false)\,true)\,(\lambda x\,.\,\lambda y\,.\,xy) \\
&= (\lambda x\,.\,\lambda y\,.\,xy)(\lambda x\,.\,false)\,true \\
&= (\lambda y\,.\,(\lambda x\,.\,false)\,y)\,true \\
&= (\lambda x\,.\,false)\,true \\
&= false.
\end{aligned}$$

- Для всех ненулевых n функция $is_zero?$ возвращает $false$.

Сложение и умножение

- Функции сложения и умножения чисел можно определить следующим образом:

$$\begin{aligned} m + n &= \lambda x\,.\,\lambda y\,.\,mx(nxy); \\ m * n &= \lambda x\,.\,\lambda y\,.\,m(nx)y. \end{aligned}$$

- Чтобы сложить m и n, нужно n раз прибавить единицу к нулю (применить x к y), а затем прибавить к полученному результату единицу еще m раз; чтобы умножить m на n, достаточно m раз прибавить n единиц к нулю.
- Интересно, что функция предшествования pred и операция вычитания определяются гораздо более сложным образом. Идея этого определения принадлежит ученику Чёрча *Стивену Клини* (1909—1994). Суть его идеи заключается в использовании вспомогательной функции, которая любой паре вида (m, n), где m и n — нумералы Чёрча, ставит в соответствие пару $(n, n + 1)$.

$$\text{step} = \lambda p\,.\,(\text{snd}\ p, \text{succ}(\text{snd}\ p)).$$

- Здесь используется введённое ранее обозначение пары, функции доступа к второму элементу пары и функция следования, однако понятно, что определение можно записать и непосредственно в виде λ-терма.

Вычитание

- Для того чтобы найти число, предшествующее заданному m, достаточно m раз применить функцию step к паре (0, 0), после этого первый компонент получившейся пары будет равен в точности $m - 1$:

$$\mathrm{pred} = \lambda m \,.\, \mathrm{fst}(m\ \mathrm{step}\ (0,0)).$$

- Применённая к нулю, функция pred даёт ноль. Пользуясь этой функцией, можно определить вычитание:

$$m - n = n\ \mathrm{pred}\ m.$$



Локальные объявления

- Язык программирования на основе λ-исчисления построен.
- Любая программа на этом языке — это λ-абстракция.
- Запуск программы означает, что этой абстракции передаётся в качестве параметра исходное значение.
- Результат редукции — результат выполнения программы.

Таким результатом оказывается некоторое выражение в слабой нормальной форме. Чтобы программирование на этом языке стало еще более удобным, добавим возможность создания локальных объявлений, то есть способ именования вспомогательных конструкций.

- Введём в рассмотрение следующее обозначение:

$$(\text{let } x = s \text{ in } t) = (\lambda x \,.\, t)\, s$$

$$\begin{aligned} (\text{let } z = 2 + 3 \text{ in } z * z) &= (\lambda z \,.\, z * z)\,(2 + 3) \\ &= (2 + 3) * (2 + 3) \end{aligned}$$

Конструкцию let/in можно использовать последовательно:

$$\text{let } x = 1 \text{ in } (\text{let } x = 2 \text{ in } (\text{let } y = x \text{ in } x + y)) = 4$$

Локальные объявления

Можно также определить параллельное объявление нескольких имён:

$$\begin{aligned}&\text{let}\\&\quad x_1 = s_1\\&\quad x_2 = s_2\\&\quad \ldots\\&\quad x_n = s_n\\&\text{in } t = (\lambda x_1 \,.\, \lambda x_2 \,.\, \ldots \lambda x_n \,.\, t)\, s_1 s_2 \ldots s_n\end{aligned}$$

- Пример его использования:

$$\begin{aligned}&\text{let } x = 1 \text{ in}\\&\qquad\qquad\text{let}\\&\qquad\qquad\quad x = 2\\&\qquad\qquad\quad y = x\\&\qquad\qquad\text{in } x + y = 3\end{aligned}$$

СПАСИБО
ЗА ВНИМАНИЕ